CODING STANDARD REFERENCE

MISRA C:2023 & C++:2023 Guidelines

A consolidated, paraphrased reference of the MISRA C and C++ coding guidelines — every directive and rule with its category, decidability, rationale, and compliant / non-compliant examples.

~221
MISRA C:2023 guidelines

179
MISRA C++:2023 guidelines

3
categories · Mandatory / Required / Advisory

Copyright & accuracy. MISRA® is a registered trademark of HORIBA MIRA Ltd. This document contains **paraphrased summaries only** — not the official wording — and is not a substitute for the purchased standard. Entries flagged `⚑verify` (amendment-added C rules; all C++ rule IDs) should be confirmed against the official document before use in a formal compliance matrix.

CONTENTS

1. MISRA C:2023 — Overview
2. MISRA C:2023 — Directives
3. MISRA C:2023 — Rules §1–§6
4. MISRA C:2023 — Rules §7–§9
5. MISRA C:2023 — Rules §10–§11
6. MISRA C:2023 — Rules §12–§16
7. MISRA C:2023 — Rules §17–§19
8. MISRA C:2023 — Rules §20
9. MISRA C:2023 — Rules §21–§22
10. MISRA C++:2023 — Overview
11. MISRA C++:2023 — Directives

12. MISRA C++:2023 — Rules

MISRA C

MISRA C:2023 — Overview

MISRA C:2023 is the third edition of the MISRA C guidelines. It consolidates **MISRA C:2012** with **Amendments 1–4** and the **Technical Corrigenda** into a single document, and covers the **C90, C99, C11 and C18** language editions.

- **~221 guidelines total** ≈ **~21 directives** + **~200 rules**.
- Each guideline has a **category** and a **decidability**.
- Rules are numbered `Rule <section>.<index>`; directives `Dir <section>.<index>`.

CATEGORIES

CATEGORY	DEVIATIONS	NOTES
Mandatory	**Never** permitted	Introduced in C:2012; violation = defect
Required	Permitted, formally recorded	The bulk of the ruleset
Advisory	Permitted, recording recommended	Good practice

RULES VS. DIRECTIVES

- **Rules** — compliance can be checked purely from the **source code**. Most are amenable to static analysis.
- **Directives** — compliance also depends on information **outside the code** (requirements, documentation, process). A tool alone cannot fully check them.

DECIDABILITY

- **Decidable** — an analyser can always give a correct yes/no.
- **Undecidable** — cannot be resolved in every case; needs review, testing, or runtime checks.
- Analysis scope is either **Single Translation Unit** or **System** (whole program / all TUs linked together).

THE ESSENTIAL TYPE MODEL (KEY CONCEPT FOR §10)

MISRA defines an "essential type" for every expression, independent of the C type used to store it. The essential type categories are:

- essentially **Boolean**
- essentially **character**
- essentially **signed** integer
- essentially **unsigned** integer
- essentially **enum** (named, non-anonymous)
- essentially **floating**

Section 10 rules constrain mixing these categories and prevent implicit conversions that silently change value or category. This model is the single most important concept to understand when applying MISRA C.

DEVIATION PROCESS (FOR REQUIRED/ADVISORY ONLY)

A compliant project may deviate from a **Required** or **Advisory** guideline if it records a **deviation**: the guideline ID, the rationale, the scope, and sign -off. **Mandatory** guidelines can never be deviated. A **Guideline Compliance Summary (GCS)** documents, for every guideline, whether the project complies, deviates, or the guideline is disapplied.

SECTION MAP (RULES)

§	TOPIC	INDEX FILE
1	A standard C environment	rules/01-06-environment-and-types.md
2	Unused code	rules/01-06-environment-and-types.md
3	Comments	rules/01-06-environment-and-types.md
4	Character sets & lexical	rules/01-06-environment-and-types.md
5	Identifiers	rules/01-06-environment-and-types.md
6	Types	rules/01-06-environment-and-types.md
7	Literals and constants	rules/07-09-constants-declarations-init.md
8	Declarations and definitions	rules/07-09-constants-declarations-init.md
9	Initialization	rules/07-09-constants-declarations-init.md
10	The essential type model	rules/10-11-types-and-pointers.md
11	Pointer type conversions	rules/10-11-types-and-pointers.md
12	Expressions	rules/12-16-expressions-and-control.md
13	Side effects	rules/12-16-expressions-and-control.md
14	Control statement expressions	rules/12-16-expressions-and-control.md
15	Control flow	rules/12-16-expressions-and-control.md
16	Switch statements	rules/12-16-expressions-and-control.md
17	Functions	rules/17-19-functions-pointers-storage.md
18	Pointers and arrays	rules/17-19-functions-pointers-storage.md
19	Overlapping storage	rules/17-19-functions-pointers-storage.md
20	Preprocessing directives	rules/20-preprocessing.md
21	Standard libraries	rules/21-22-libraries-and-resources.md
22	Resources	rules/21-22-libraries-and-resources.md

Directives live in `directives.md`.

> `⚑verify` on an entry means: check the ID/category against the official standard before using it in a formal compliance record — it was added or changed by an amendment and public reconstructions vary.

MISRA C

MISRA C:2023 — Directives (in depth)

Directives require information **beyond the source code** (requirements, documentation, build process) to check. Each entry: **ID — Category · Decidability · Scope**, paraphrased guideline, rationale, and guidance.

DIR 1 — THE IMPLEMENTATION

Dir 1.1 — Required · Undecidable · System

Guideline: Any implementation-defined behaviour the program's output depends on must be documented and understood. **Rationale:** Type sizes, `char` signedness, right-shift of negatives, etc. vary by compiler; relying on them undocumented breaks portability.

Dir 1.2 — Advisory ⚑verify

Guideline: Language extensions should be documented and understood.

DIR 2 — COMPILATION AND BUILD

Dir 2.1 — Required · Undecidable · System

Guideline: All source files must compile without any compilation errors. **Rationale:** Warnings/errors suppressed or ignored hide real defects; a clean build at max diagnostic level is the baseline.

DIR 3 — REQUIREMENTS TRACEABILITY

Dir 3.1 — Required · Undecidable · System

Guideline: All code must be traceable to documented requirements. **Rationale:** Untraceable code may be unnecessary or unverified — a safety concern.

DIR 4 — CODE DESIGN

Dir 4.1 — Required · Undecidable · System

Guideline: Run-time failures must be minimized. **Rationale:** Guard against division by zero, overflow, out-of-range indexing, null dereference — by design, static proof, or run-time checks.

Dir 4.2 — Advisory · Undecidable · Single TU

Guideline: All use of assembly language should be documented.

Dir 4.3 — Required · Decidable · Single TU

Guideline: Assembly language must be encapsulated and isolated.

```
/* Compliant: wrap asm in a documented inline function */
static inline void nop(void){ __asm("nop"); }
```

Dir 4.4 — Advisory · Undecidable · Single TU

Guideline: Sections of code should not be "commented out". **Rationale:** Use version control, not comments, to remove code. Commented-out code rots and confuses.

Dir 4.5 — Advisory · Undecidable · System

Guideline: Identifiers in the same name space with overlapping visibility should be typographically unambiguous (avoid `l` / `1`, `O` / `0`, `rn` / `m`).

Dir 4.6 — Advisory · Undecidable · Single TU

Guideline: `typedef`s that indicate size and signedness should be used in place of the basic numerical types.

```c
int   count;     /* Non-compliant */
int32_t count;   /* Compliant */
```

Dir 4.7 — Required · Undecidable · Single TU

Guideline: If a function returns error information, that error information must be tested.

```c
(void)fclose(f);          /* Non-compliant if the result matters */
if (fclose(f) != 0) { /* handle */ }   /* Compliant */
```

Dir 4.8 — Advisory · Undecidable · Single TU

Guideline: If a pointer to a struct/union is never dereferenced within a TU, its implementation should be hidden (opaque type).

Dir 4.9 — Advisory · Undecidable · Single TU

Guideline: A function should be preferred over a function-like macro where interchangeable. **Rationale:** Functions are type-checked and side-effect-safe; macros aren't.

Dir 4.10 — Required · Decidable · Single TU

Guideline: Precautions must be taken to prevent a header being included more than once (include guards / `#pragma once`).

```c
#ifndef FOO_H
#define FOO_H
/* ... */
#endif
```

Dir 4.11 — Required · Undecidable · Single TU

Guideline: The validity of values passed to library functions must be checked.

```c
if (x >= 0.0) { r = sqrt(x); }   /* Compliant: domain checked */
```

Dir 4.12 — Required · Undecidable · Single TU

Guideline: Dynamic memory allocation must not be used. Rationale: Non-deterministic timing, fragmentation, and leak/UAF risk make it unsuitable for many safety-critical systems.

Dir 4.13 — Advisory · Undecidable · Single TU

Guideline: Functions operating on a resource should be called in an appropriate sequence (e.g. open → use → close).

Dir 4.14 — Required · Undecidable · Single TU *(AMD1 — security)*

Guideline: The validity of values received from external sources must be checked (bounds, type, range) before use.

Dir 4.15 — Required · Undecidable · Single TU ⚑verify *(AMD3)*

Guideline: Evaluation of floating-point expressions must not lead to the undetected generation of infinities and NaNs.

DIR 5 — CONCURRENCY *(C:2023 AMENDMENTS)* ⚑VERIFY

The published C:2023 total is ~21 directives across ~5 sections (vs 17 in the original C:2012). A **concurrency** directive area (`Dir 5.x`), tied to the C11 threading additions, is associated with the newer editions. **Confirm the exact IDs, titles, and categories of any `Dir 5.x` guidelines against the official MISRA C:2023 document** before citing them in a compliance matrix.

MISRA C

MISRA C:2023 — Rules §1–§6 (in depth)

Each entry: **ID — Category · Decidability · Scope**, paraphrased guideline, rationale, and (where useful) non-compliant vs compliant C. Summaries are paraphrased; verify `⚑verify` entries against the official standard.

---

§1 — A STANDARD C ENVIRONMENT

Rule 1.1 — Required · Decidable · System

Guideline: The program must contain no violations of standard C syntax and constraints, and must not exceed the implementation's translation limits. **Rationale:** Code relying on non-standard syntax or exceeding documented limits (nesting depth, identifier length, etc.) may be rejected or silently mistranslated by a conforming compiler.

```
/* Non-compliant: constraint violation — assigning to array name */
int a[4]; a = other;        /* not an lvalue */
/* Compliant */
int a[4]; (void)memcpy(a, other, sizeof a);
```

Rule 1.2 — Advisory · Undecidable · System

Guideline: Language extensions should not be used. **Rationale:** Extensions (`__attribute__`, `#pragma`, inline asm keywords) are not portable and change meaning between compilers.

```
int x __attribute__((aligned(16)));   /* Non-compliant: GCC extension */
```

Exception: Extensions needed for the target may be used if isolated and documented (see Dir 4.3 / Dir 1.2).

Rule 1.3 — Required · Undecidable · System

Guideline: There must be no occurrence of undefined or critical unspecified behaviour. **Rationale:** UB (signed overflow, out-of-bounds access, use-after-free, data races) makes program behaviour unpredictable across compilers/optimisation levels.

```
int i = INT_MAX; i = i + 1;   /* Non-compliant: signed overflow is UB */
```

Rule 1.4 — Required · Decidable · Single TU ⚑verify *(AMD2, C11/C18)*

Guideline: Emergent (newer) language features must not be used. **Rationale:** Features such as `_Generic`, `_Atomic`, `_Noreturn`, `<threads.h>` carry portability and analysis risks; restrict them unless explicitly adopted.

Rule 1.5 — Required ⚑verify *(AMD3)*

Guideline: Obsolescent language features must not be used. **Rationale:** Features flagged obsolescent by the C standard (e.g. certain `gets`-style usages) may be removed and are error-prone.

---

§2 — UNUSED CODE

Rule 2.1 — Required · Undecidable · System

Guideline: A project must not contain unreachable code. **Rationale:** Unreachable code cannot execute; its presence signals a logic error and bloats the binary.

```
return x;
y = 1;          /* Non-compliant: unreachable */
```

Rule 2.2 — Required · Undecidable · System

Guideline: There must be no dead code (operations whose removal cannot affect behaviour). **Rationale:** Dead code wastes effort, confuses reviewers, and often hides a bug.

```
(void)(a + b);  /* Non-compliant: result unused, no side effect */
```

Rule 2.3 — Advisory · Decidable · System

Guideline: A project should not contain unused type declarations. **Rationale:** Stale `typedef`s/tags clutter the code and mislead maintainers.

Rule 2.4 — Advisory · Decidable · System

Guideline: A project should not contain unused tag declarations.

Rule 2.5 — Advisory · Decidable · System

Guideline: A project should not contain unused macro declarations.

Rule 2.6 — Advisory · Decidable · Single TU

Guideline: A function should not contain unused label declarations.

Rule 2.7 — Advisory · Decidable · Single TU

Guideline: There should be no unused parameters in functions.

```
void f(int used, int unused) { g(used); }   /* Non-compliant */
void f(int used, int x)      { (void)x; g(used); }  /* Compliant: explicitly consumed
```

Rule 2.8 — Advisory · Decidable · System ⚑verify *(AMD)*

Guideline: A project should not contain unused object definitions.

---

§3 — COMMENTS

Rule 3.1 — Required · Decidable · Single TU

Guideline: The sequences `/*` and `//` must not appear within a comment.
Rationale: Nested comment markers cause code to be unexpectedly commented out.

```
/* opening /* nested */ x = 1;   /* Non-compliant: x=1 is NOT commented out as intend
```

Rule 3.2 — Required · Decidable · Single TU

Guideline: Line-splicing (`\` at end of line) must not be used in `//` comments.
Rationale: A trailing backslash silently extends a `//` comment onto the next line, swallowing code.

```
// comment ending in backslash \
next_statement();   /* Non-compliant: this line is part of the comment */
```

---

§4 — CHARACTER SETS AND LEXICAL CONVENTIONS

Rule 4.1 — Required · Decidable · Single TU

Guideline: Octal and hexadecimal escape sequences must be terminated. **Rationale:** `"\x1f2"` is one 3-digit hex escape, not `\x1f` + `2` — terminate to avoid ambiguity.

```
char s[] = "\x1B[0m";        /* Non-compliant: \x1B0 ambiguous */
char s[] = "\x1B" "[0m";     /* Compliant: adjacent literals terminate the escape */
```

Rule 4.2 — Advisory · Decidable · Single TU

Guideline: Trigraphs should not be used. **Rationale:** `??=` etc. are replaced during translation and cause surprising edits to string contents.

---

§5 — IDENTIFIERS

Rule 5.1 — Required · Decidable · System

Guideline: External identifiers must be distinct (within the significant- character limit). **Rationale:** Two externals identical in the first N characters clash at link time on limited toolchains.

Rule 5.2 — Required · Decidable · Single TU

Guideline: Identifiers in the same scope and name space must be distinct.

Rule 5.3 — Required · Decidable · Single TU

Guideline: An identifier in an inner scope must not hide one in an outer scope.

```
int count;
void f(void){ int count = 0; }   /* Non-compliant: shadows file-scope count */
```

Rule 5.4 — Required · Decidable · Single TU

Guideline: Macro identifiers must be distinct.

Rule 5.5 — Required · Decidable · Single TU

Guideline: Identifiers must be distinct from macro names.

Rule 5.6 — Required · Decidable · System

Guideline: A `typedef` name must be a unique identifier.

Rule 5.7 — Required · Decidable · System

Guideline: A tag name must be a unique identifier.

Rule 5.8 — Required · Decidable · System

Guideline: Identifiers with external linkage that define objects/functions must be unique.

Rule 5.9 — Advisory · Decidable · System

Guideline: Identifiers with internal linkage that define objects/functions should be unique.

---

§6 — TYPES

Rule 6.1 — Required · Decidable · Single TU

Guideline: Bit-fields must only be declared with an appropriate type. **Rationale:** The set of permitted bit-field types is implementation-defined for types other than `int` / `unsigned int` / `_Bool`; use only portable ones.

```
struct S { char b : 2; };        /* Non-compliant: char bit-field not portable */
struct S { unsigned int b : 2; };/* Compliant */
```

Rule 6.2 — Required · Decidable · Single TU

Guideline: Single-bit named bit-fields must not be of a signed type. **Rationale:** A 1-bit signed field holds only 0 and −1, almost never what's meant.

```
struct S { signed int flag : 1; };   /* Non-compliant */
struct S { unsigned int flag : 1; }; /* Compliant */
```

Rule 6.3 — Required · Decidable · Single TU ⚑verify *(AMD)*

Guideline: A bit-field must not be declared as a member of a union.

MISRA C

MISRA C:2023 — Rules §7–§9 (in depth)

Each entry: **ID — Category · Decidability · Scope**, paraphrased guideline, rationale, and examples. `⚑verify` = confirm against the official standard.

---

§7 — LITERALS AND CONSTANTS

Rule 7.1 — Required · Decidable · Single TU

Guideline: Octal constants must not be used. **Rationale:** A leading `0` silently makes a constant octal — a classic bug.

```
int mask = 011;   /* Non-compliant: octal 011 == decimal 9, not 11 */
int mask = 9;     /* Compliant */
```

Exception: The single digit `0` is not an octal constant.

Rule 7.2 — Required · Decidable · Single TU

Guideline: A `u` / `U` suffix must be applied to all integer constants represented in an unsigned type. **Rationale:** Makes the constant's signedness explicit and avoids surprising signed/unsigned conversions.

```
uint32_t x = 0xFFFFFFFF;    /* Non-compliant: value needs unsigned type */
uint32_t x = 0xFFFFFFFFU;   /* Compliant */
```

Rule 7.3 — Required · Decidable · Single TU

Guideline: The lowercase `l` must not be used in a literal suffix. **Rationale:** `l` is easily confused with `1`. Use `L`.

```
long n = 100l;   /* Non-compliant: looks like 1001 */
long n = 100L;   /* Compliant */
```

Rule 7.4 — Required · Decidable · Single TU

Guideline: A string literal must not be assigned to an object unless its type is pointer to const-qualified `char`. **Rationale:** String literals are non-modifiable; writing through a non-const pointer is UB.

```
char *p = "hello";        /* Non-compliant */
const char *p = "hello";  /* Compliant */
```

Rule 7.5 — Required ⚑verify *(AMD3)*

Guideline: An integer constant macro argument must have the type its macro requires.

Rule 7.6 — Required ⚑verify *(AMD3)*

Guideline: The small-integer variants of the minimum-width integer constant macros must not be used.

---

§8 — DECLARATIONS AND DEFINITIONS

Rule 8.1 — Required · Decidable · Single TU

Guideline: Types must be explicitly specified (no implicit `int`).

```
static x;         /* Non-compliant: implicit int */
static int x;     /* Compliant */
```

Rule 8.2 — Required · Decidable · Single TU

Guideline: Function types must be in prototype form with named parameters.

```
int f();              /* Non-compliant: not a prototype */
int f(int a, int b);  /* Compliant */
```

Rule 8.3 — Required · Decidable · System

Guideline: All declarations of an object/function must use the same names and type qualifiers. **Rationale:** Divergent declarations across TUs cause silent type mismatches.

Rule 8.4 — Required · Decidable · System

Guideline: A compatible declaration must be visible when an object/function with external linkage is defined. **Rationale:** Forces the definition to be checked against the header prototype.

```
/* foo.c — Non-compliant: no visible prototype for exported symbol */
int foo(void){ return 1; }
/* Compliant: include the header that declares foo */
#include "foo.h"
int foo(void){ return 1; }
```

Rule 8.5 — Required · Decidable · System

Guideline: An external object/function must be declared once, in one file (typically a header).

Rule 8.6 — Required · Decidable · System

Guideline: An identifier with external linkage must have exactly one external definition.

Rule 8.7 — Advisory · Decidable · System

Guideline: Functions/objects should not have external linkage if referenced in only one TU (make them `static`).

Rule 8.8 — Required · Decidable · Single TU

Guideline: `static` must be used in all declarations of objects/functions with internal linkage.

Rule 8.9 — Advisory · Decidable · System

Guideline: An object should be defined at block scope if used in only one function.

Rule 8.10 — Required · Decidable · Single TU

Guideline: An inline function must be declared `static`. **Rationale:** Avoids the subtle external-inline linkage rules of C99+.

Rule 8.11 — Advisory · Decidable · Single TU

Guideline: When an array with external linkage is declared, its size should be specified explicitly.

```
extern int table[];    /* Non-compliant */
extern int table[10];  /* Compliant */
```

Rule 8.12 — Required · Decidable · Single TU

Guideline: Within an enumerator list, implicitly-specified enum constant values must be unique.

```
enum E { A = 1, B, C = 2 };   /* Non-compliant: B and C both == 2 */
```

Rule 8.13 — Advisory · Undecidable · System

Guideline: A pointer should point to a const-qualified type whenever possible.

Rule 8.14 — Required · Decidable · Single TU

Guideline: The `restrict` type qualifier must not be used. **Rationale:** Misuse silently introduces UB via wrong aliasing assumptions.

Rule 8.15–8.17 — ⚑verify *(AMD2, C11/C18)*

Guideline: Constraints on alignment specifiers (`_Alignas`, `_Alignof`) and over-aligned types.

---

§9 — INITIALIZATION

Rule 9.1 — Mandatory · Undecidable · System

Guideline: The value of an automatic object must not be read before it is set. **Rationale:** Reading an indeterminate value is UB. **No deviation permitted.**

```
int x; use(x);        /* Non-compliant */
int x = 0; use(x);    /* Compliant */
```

Rule 9.2 — Required · Decidable · Single TU

Guideline: The initializer for an aggregate or union must be enclosed in braces.

```
int m[2][2] = {1,2,3,4};        /* Non-compliant */
int m[2][2] = {{1,2},{3,4}};    /* Compliant */
```

Rule 9.3 — Required · Decidable · Single TU

Guideline: Arrays must not be partially initialized.

```
int a[3] = {1};        /* Non-compliant: a[1],a[2] implicitly 0 — state intent */
int a[3] = {1, 0, 0};  /* Compliant */
```

Exception: `= {0}` to zero the whole array is permitted.

Rule 9.4 — Required · Decidable · Single TU

Guideline: An element of an object must not be initialized more than once.

```
int a[3] = {[0]=1, [0]=2};   /* Non-compliant */
```

Rule 9.5 — Required · Decidable · Single TU

Guideline: Where designated initializers initialize an array, the array size must be specified explicitly.

```
int a[]  = {[2]=1};    /* Non-compliant: size implicit */
int a[3] = {[2]=1};    /* Compliant */
```

Rule 9.6 — Required ⚑verify *(AMD3)*

Guideline: An initializer using chained designators must not mix in initializers without designators.

Rule 9.7 — Mandatory ⚑verify *(AMD2/AMD3, C11)*

Guideline: Atomic objects must be appropriately initialized before access.

MISRA C

MISRA C:2023 — Rules §10–§11 (in depth)

Each entry: **ID — Category · Decidability · Scope**, paraphrased guideline, rationale, and examples. See `../overview.md` for the essential type model.

---

§10 — THE ESSENTIAL TYPE MODEL

The "essential type" of an expression is its natural type before implicit promotion. Categories: Boolean, character, signed, unsigned, enum, floating. These rules stop implicit mixing that silently changes value or category.

Rule 10.1 — Required · Decidable · Single TU

Guideline: Operands must not be of an inappropriate essential type for the operator.

```
char c; int x = c << 1;    /* Non-compliant: shift on essentially char */
```

Rule 10.2 — Required · Decidable · Single TU

Guideline: Expressions of essentially character type must not be used inappropriately in `+` / `-`.

```
char c = '0' + '5';        /* Non-compliant: adding two chars */
char c = (char)('0' + 5);  /* Compliant: char + int(offset) */
```

Rule 10.3 — Required · Decidable · Single TU

Guideline: A value must not be assigned to an object with narrower essential type or a different essential type category.

```
uint8_t b = 300;           /* Non-compliant: narrows, and 300 > 255 */
int16_t s = -1; uint16_t u = s;  /* Non-compliant: category change signed->unsigned */
```

Rule 10.4 — Required · Decidable · Single TU

Guideline: Both operands of an operator using the usual arithmetic conversions must have the same essential type category.

```
uint32_t u; int32_t s; if (u > s) {}   /* Non-compliant: unsigned vs signed */
```

Rule 10.5 — Advisory · Decidable · Single TU

Guideline: A value should not be cast to an inappropriate essential type.

```
(bool)3.0f;    /* Non-compliant: floating cast to bool */
```

Rule 10.6 — Required · Decidable · Single TU

Guideline: The value of a composite expression must not be assigned to an object with wider essential type.

```
uint16_t a,b; uint32_t r = a + b;   /* Non-compliant: a+b is 16-bit composite */
uint32_t r = (uint32_t)a + b;       /* Compliant */
```

Rule 10.7 — Required · Decidable · Single TU

Guideline: If a composite expression is one operand where usual arithmetic conversions apply, the other operand must not have wider essential type.

Rule 10.8 — Required · Decidable · Single TU

Guideline: The value of a composite expression must not be cast to a different essential type category or a wider essential type.

---

§11 — POINTER TYPE CONVERSIONS

Rule 11.1 — Required · Decidable · Single TU

Guideline: Conversions must not be performed between a function pointer and any other type.

```
void (*fp)(void) = (void(*)(void))0x8000;  /* Non-compliant */
```

Rule 11.2 — Required · Decidable · Single TU

Guideline: Conversions must not be performed between a pointer to an incomplete type and any other type.

Rule 11.3 — Required · Decidable · Single TU

Guideline: A cast must not be performed between a pointer to object type and a pointer to a different object type.

```
uint8_t *p; uint32_t *q = (uint32_t*)p;   /* Non-compliant: alignment/aliasing risk */
```

Rule 11.4 — Advisory · Decidable · Single TU

Guideline: A conversion should not be performed between a pointer to object and an integer type.

```
uintptr_t a = (uintptr_t)ptr;   /* Non-compliant (advisory): document if required */
```

Rule 11.5 — Advisory · Decidable · Single TU

Guideline: A conversion should not be performed from pointer to `void` into pointer to object.

Rule 11.6 — Required · Decidable · Single TU

Guideline: A cast must not be performed between pointer to `void` and an arithmetic type.

Rule 11.7 — Required · Decidable · Single TU

Guideline: A cast must not be performed between pointer to object and a non-integer arithmetic type.

Rule 11.8 — Required · Decidable · Single TU

Guideline: A cast must not remove `const` / `volatile` qualification from the pointed-to type.

```
const int *cp; int *p = (int*)cp;   /* Non-compliant: casts away const */
```

Rule 11.9 — Required · Decidable · Single TU

Guideline: The macro `NULL` must be the only permitted form of integer null pointer constant.

```
int *p = 0;      /* Non-compliant */
int *p = NULL;   /* Compliant */
```

Rule 11.10 — Required ⚑verify *(AMD3, C11)*

Guideline: The `_Atomic` qualifier must not be applied to incomplete `void`.

MISRA C

MISRA C:2023 — Rules §12–§16 (in depth)

Each entry: **ID — Category · Decidability · Scope**, paraphrased guideline, rationale, and examples.

---

§12 — EXPRESSIONS

Rule 12.1 — Advisory · Decidable · Single TU

Guideline: Operator precedence within expressions should be made explicit.

```
x = a + b << c;         /* Non-compliant: is it (a+b)<<c ? */
x = (a + b) << c;       /* Compliant */
```

Rule 12.2 — Required · Decidable · Single TU

Guideline: The right operand of a shift must lie in 0 .. (bit-width of left − 1).

```
uint16_t v; v = v << 20;    /* Non-compliant: shift >= width is UB */
```

Rule 12.3 — Advisory · Decidable · Single TU

Guideline: The comma operator should not be used.

```
for (i=0, j=n; i<j; i++, j--)   /* Non-compliant */
```

Rule 12.4 — Advisory · Decidable · Single TU

Guideline: Evaluation of constant expressions should not lead to unsigned integer wrap-around.

```
uint16_t x = 0xFFFFu + 1u;   /* Non-compliant: wraps to 0 */
```

Rule 12.5 — Mandatory · Decidable · Single TU *(AMD1)*

Guideline: `sizeof` must not have an operand that is a function parameter declared as "array of type". **No deviation permitted.**

```
void f(int a[10]){ size_t n = sizeof(a); }  /* Non-compliant: a is a pointer, sizeof==
```

---

§13 — SIDE EFFECTS

Rule 13.1 — Required · Undecidable · Single TU

Guideline: Initializer lists must not contain persistent side effects.

```
int a[2] = { i++, i++ };   /* Non-compliant */
```

Rule 13.2 — Required · Undecidable · System

Guideline: An expression's value and side effects must be identical under all permitted evaluation orders.

```
y = f() + g();   /* Non-compliant if f and g share/modify state */
```

Rule 13.3 — Advisory · Decidable · Single TU

Guideline: A full expression with `++` / `--` should have no other potential side effects.

```
x = a++ + b--;   /* Non-compliant */
```

Rule 13.4 — Advisory · Decidable · Single TU

Guideline: The result of an assignment operator should not be used.

```
if ((x = get()) != 0) {}   /* Non-compliant */
x = get(); if (x != 0) {}  /* Compliant */
```

Rule 13.5 — Required · Undecidable · Single TU

Guideline: The right operand of `&&` / `||` must not contain persistent side effects.

```
if (ok && (count++ > 0)) {}   /* Non-compliant: count++ may not run */
```

Rule 13.6 — Mandatory · Decidable · Single TU

Guideline: The operand of `sizeof` must not contain any expression with potential side effects. **No deviation permitted.**

```
n = sizeof(a[i++]);   /* Non-compliant: i++ not evaluated */
```

---

§14 — CONTROL STATEMENT EXPRESSIONS

Rule 14.1 — Required · Undecidable · Single TU

Guideline: A loop counter must not have essentially floating type.

```
for (float f=0.0f; f!=1.0f; f+=0.1f) {}   /* Non-compliant: never exactly 1.0 */
```

Rule 14.2 — Required · Undecidable · Single TU

Guideline: A `for` loop must be well-formed (single counter, clear init/cond/ step, counter not modified in body).

Rule 14.3 — Required · Undecidable · System

Guideline: Controlling expressions must not be invariant.

```
if (sizeof(int) > 0) {}   /* Non-compliant: always true */
```

Rule 14.4 — Required · Decidable · Single TU

Guideline: The controlling expression of `if`/iteration must have essentially Boolean type.

```
int p; if (p) {}          /* Non-compliant */
int p; if (p != 0) {}     /* Compliant */
```

---

§15 — CONTROL FLOW

Rule 15.1 — Advisory · Decidable · Single TU

Guideline: `goto` should not be used.

Rule 15.2 — Required · Decidable · Single TU

Guideline: `goto` must jump to a label later in the same function.

```
loop: ... goto loop;   /* Non-compliant: backward jump */
```

Rule 15.3 — Required · Decidable · Single TU

Guideline: A label referenced by `goto` must be in the same or an enclosing block of the `goto`.

Rule 15.4 — Advisory · Decidable · Single TU

Guideline: No more than one `break`/`goto` should terminate any iteration statement.

Rule 15.5 — Advisory · Decidable · Single TU

Guideline: A function should have a single point of exit at the end.

```
int f(int x){ if(x) return 1; return 0; }   /* Non-compliant: two returns */
```

Rule 15.6 — Required · Decidable · Single TU

Guideline: The body of an iteration/selection statement must be a compound statement `{ }`.

```
if (x) y = 1;              /* Non-compliant */
if (x) { y = 1; }         /* Compliant */
```

Rule 15.7 — Required · Decidable · Single TU

Guideline: All `if ... else if` chains must terminate with an `else`.

```
if (a) {} else if (b) {}              /* Non-compliant */
if (a) {} else if (b) {} else {}     /* Compliant */
```

---

§16 — SWITCH STATEMENTS

Rule 16.1 — Required · Decidable · Single TU

Guideline: All `switch` statements must be well-formed (per rules 16.2–16.7).

Rule 16.2 — Required · Decidable · Single TU

Guideline: A switch label must only appear when the enclosing compound statement is the switch body (no Duff's-device-style labels).

Rule 16.3 — Required · Decidable · Single TU

Guideline: An unconditional `break` must terminate every switch-clause.

```
case 1: x=1;           /* Non-compliant: falls through */
case 1: x=1; break;    /* Compliant */
```

Rule 16.4 — Required · Decidable · Single TU

Guideline: Every `switch` must have a `default` label.

Rule 16.5 — Required · Decidable · Single TU

Guideline: A `default` label must be the first or last switch label.

Rule 16.6 — Required · Decidable · Single TU

Guideline: Every `switch` must have at least two switch-clauses. **Rationale:** A one-clause switch should be an `if`.

Rule 16.7 — Required · Decidable · Single TU

Guideline: A switch-expression must not have essentially Boolean type (use `if` / `else`).

MISRA C

MISRA C:2023 — Rules §17–§19 (in depth)

Each entry: **ID — Category · Decidability · Scope**, paraphrased guideline, rationale, and examples. `⚑verify` = confirm against the official standard.

---

§17 — FUNCTIONS

Rule 17.1 — Required · Decidable · Single TU

Guideline: The features of `<stdarg.h>` (`va_list`, `va_start`, …) must not be used. **Rationale:** Variadic functions bypass type checking.

Rule 17.2 — Required · Undecidable · System

Guideline: Functions must not call themselves, directly or indirectly. **Rationale:** Recursion makes stack usage unbounded/unanalysable.

```
int fact(int n){ return n?n*fact(n-1):1; }   /* Non-compliant */
```

Rule 17.3 — Mandatory · Decidable · Single TU

Guideline: A function must not be declared implicitly. **No deviation.**

```
y = foo();   /* Non-compliant if no prototype for foo is visible */
```

Rule 17.4 — Mandatory · Decidable · Single TU

Guideline: All exit paths from a non-`void` function must have an explicit `return` with an expression. **No deviation.**

```
int f(int x){ if(x) return 1; }   /* Non-compliant: falls off the end */
```

Rule 17.5 — Advisory · Undecidable · Single TU

Guideline: The argument for a parameter declared with an array type must have an appropriate number of elements.

Rule 17.6 — Mandatory · Decidable · Single TU

Guideline: An array parameter declaration must not contain `static` between the `[ ]`. **No deviation.**

```
void f(int a[static 10]);   /* Non-compliant */
```

Rule 17.7 — Required · Decidable · Single TU

Guideline: The value returned by a non-`void` function must be used.

```
foo();              /* Non-compliant if foo returns a value */
(void)foo();        /* Compliant: explicitly discarded */
```

Rule 17.8 — Advisory · Decidable · Single TU

Guideline: A function parameter should not be modified.

```
void f(int n){ n++; use(n); }        /* Non-compliant */
void f(int n){ int m=n+1; use(m); }  /* Compliant */
```

Rule 17.9 — Mandatory ⚑verify *(AMD2, C11)*

Guideline: A `_Noreturn` function must not return to its caller.

Rule 17.10 — Required ⚑verify *(AMD2)*

Guideline: A `_Noreturn` function must have `void` return type.

Rule 17.11 — Advisory ⚑verify *(AMD2)*

Guideline: A function that never returns should be declared `_Noreturn`.

Rule 17.12 — Advisory ⚑verify *(AMD3)*

Guideline: A function identifier should be used only with `&` or a parenthesised argument list.

Rule 17.13 — Required ⚑verify *(AMD3)*

Guideline: A function type must not be type qualified.

---

§18 — POINTERS AND ARRAYS

Rule 18.1 — Required · Undecidable · Single TU

Guideline: A pointer from arithmetic on a pointer operand must address an element of the same array.

```
int a[10]; int *p = &a[0] + 11;   /* Non-compliant: out of bounds */
```

Rule 18.2 — Required · Undecidable · Single TU

Guideline: Pointer subtraction must only apply to pointers into the same array.

Rule 18.3 — Required · Undecidable · Single TU

Guideline: `>`, `>=`, `<`, `<=` must not be applied to pointers unless they point into the same object.

Rule 18.4 — Advisory · Decidable · Single TU

Guideline: `+`, `-`, `+=`, `-=` should not be applied to pointer expressions (prefer array indexing).

```
p += 1;      /* Non-compliant (advisory) */
x = p[1];    /* Compliant */
```

Rule 18.5 — Advisory · Decidable · Single TU

Guideline: Declarations should contain no more than two levels of pointer nesting.

```
int ***p;    /* Non-compliant */
```

Rule 18.6 — Required · Undecidable · System

Guideline: The address of an object with automatic storage must not be copied to an object that persists after it ceases to exist.

```
int *g;
void f(void){ int local; g = &local; }   /* Non-compliant: dangling after return */
```

Rule 18.7 — Required · Decidable · Single TU

Guideline: Flexible array members must not be declared.

```
struct S { int n; int data[]; };   /* Non-compliant */
```

Rule 18.8 — Required · Decidable · Single TU

Guideline: Variable-length array types must not be used.

```
void f(int n){ int a[n]; }   /* Non-compliant: VLA */
```

Rule 18.9 — Required ⚑verify *(AMD3)*

Guideline: The address of an object with temporary lifetime must not outlive it.

Rule 18.10 — Mandatory ⚑verify *(AMD3)*

Guideline: Pointers to variably-modified array types must not be used.

---

§19 — OVERLAPPING STORAGE

Rule 19.1 — Mandatory · Undecidable · System

Guideline: An object must not be assigned or copied to an overlapping object. **No deviation.**

```
memcpy(&buf[0], &buf[1], 4);   /* Non-compliant: overlapping */
```

Rule 19.2 — Advisory · Decidable · Single TU

Guideline: The `union` keyword should not be used. **Rationale:** Reading a member other than the last written is unspecified/UB.

MISRA C

MISRA C:2023 — Rules §20 (Preprocessing directives, in depth)

Each entry: **ID — Category · Decidability · Scope**, paraphrased guideline, rationale, and examples.

Rule 20.1 — Advisory · Decidable · Single TU

Guideline: `#include` directives should only be preceded by preprocessor directives or comments. **Rationale:** Keeps include order predictable and header effects at the top.

Rule 20.2 — Required · Decidable · Single TU

Guideline: The `'`, `"`, `\` characters and the `/*`, `//` sequences must not occur in a header file name.

```
#include "sub\dir\h.h"   /* Non-compliant: \ is UB in header name */
#include "sub/dir/h.h"   /* Compliant */
```

Rule 20.3 — Required · Decidable · Single TU

Guideline: `#include` must be followed by `<filename>` or `"filename"`.

Rule 20.4 — Required · Decidable · Single TU

Guideline: A macro must not be defined with the same name as a keyword.

```
#define const   /* Non-compliant */
```

Rule 20.5 — Advisory · Decidable · Single TU

Guideline: `#undef` should not be used. **Rationale:** `#undef` obscures which definition of a macro is in force.

Rule 20.6 — Required · Decidable · Single TU

Guideline: Tokens that look like a preprocessing directive must not occur within a macro argument.

```
FN(
#include "x.h"   /* Non-compliant */
)
```

Rule 20.7 — Required · Decidable · Single TU

Guideline: Expressions resulting from expansion of macro parameters must be enclosed in parentheses.

```
#define SQ(x) x*x           /* Non-compliant: SQ(a+b) -> a+b*a+b */
#define SQ(x) ((x)*(x))     /* Compliant */
```

Rule 20.8 — Required · Decidable · Single TU

Guideline: The controlling expression of `#if` / `#elif` must evaluate to 0 or 1.

Rule 20.9 — Required · Decidable · Single TU

Guideline: All identifiers in a `#if` / `#elif` controlling expression must be `#define`d before evaluation.

```
#if FEATURE_X           /* Non-compliant if FEATURE_X may be undefined */
#if defined(FEATURE_X) && (FEATURE_X == 1)   /* Compliant */
```

Rule 20.10 — Advisory · Decidable · Single TU

Guideline: The `#` and `##` preprocessor operators should not be used. **Rationale:** Stringisation/token-pasting are hard to read and error-prone.

Rule 20.11 — Required · Decidable · Single TU

Guideline: A macro parameter immediately after `#` must not be immediately followed by `##`.

Rule 20.12 — Required · Decidable · Single TU

Guideline: A macro parameter used with `#` / `##`, and itself subject to further replacement, must only be used as an operand to these operators.

Rule 20.13 — Required · Decidable · Single TU

Guideline: A line whose first token is `#` must be a valid preprocessing directive.

Rule 20.14 — Required · Decidable · Single TU

Guideline: All `#else`, `#elif`, `#endif` must reside in the same file as the `#if` / `#ifdef` / `#ifndef` they relate to.

MISRA C

MISRA C:2023 — Rules §21–§22 (in depth)

Each entry: **ID — Category · Decidability · Scope**, paraphrased guideline, rationale, and examples. Sections 21–22 were heavily extended by Amendments 1–3 (security, C11). Amendment-added entries are marked `⚑verify`.

Rule 21.1 — Required · Decidable · Single TU

Guideline: `#define` / `#undef` must not be used on a reserved identifier/macro.

```
#define _MYGUARD   /* Non-compliant: leading underscore reserved */
```

Rule 21.2 — Required · Decidable · Single TU

Guideline: A reserved identifier or reserved macro name must not be declared.

Rule 21.3 — Required · Decidable · Single TU

Guideline: The memory allocation/deallocation functions of `<stdlib.h>` (`malloc`, `calloc`, `realloc`, `free`) must not be used. **Rationale:** Dynamic allocation risks fragmentation, leaks, and non-determinism (see Dir 4.12).

Rule 21.4 — Required · Decidable · Single TU

Guideline: `<setjmp.h>` must not be used.

Rule 21.5 — Required · Decidable · Single TU

Guideline: `<signal.h>` must not be used.

Rule 21.6 — Required · Decidable · Single TU

Guideline: The Standard Library I/O functions (`<stdio.h>`, wide I/O) must not be used.

Rule 21.7 — Required · Decidable · Single TU

Guideline: `atof`, `atoi`, `atol`, `atoll` must not be used. **Rationale:** No error reporting on malformed input; use `strtol` family.

Rule 21.8 — Required · Decidable · Single TU

Guideline: `abort`, `exit`, `getenv` (and `system`) of `<stdlib.h>` must not be used. *(scope adjusted by AMD; `system` split to 21.21)*

Rule 21.9 — Required · Decidable · Single TU

Guideline: `bsearch` and `qsort` of `<stdlib.h>` must not be used.

Rule 21.10 — Required · Decidable · Single TU

Guideline: The date/time functions of `<time.h>` (and `wcsftime`) must not be used.

Rule 21.11 — Required · Decidable · Single TU

Guideline: `<tgmath.h>` must not be used. *(⚑verify — some editions relax to Advisory)*

Rule 21.12 — Advisory · Decidable · Single TU

Guideline: The exception-handling features of `<fenv.h>` should not be used. *(⚑verify — AMD3 may widen)*

Rule 21.13 — Mandatory · Decidable · Single TU ⚑verify *(AMD1)*

Guideline: Any value passed to a `<ctype.h>` function must be representable as `unsigned char` or equal `EOF`. **No deviation.**

```
char c = fgetc_result; isdigit(c);      /* Non-compliant if char is signed & negative
isdigit((unsigned char)c);              /* Compliant */
```

Rule 21.14 — Required · Decidable · Single TU ⚑verify *(AMD1)*

Guideline: `memcmp` must not be used to compare null-terminated strings.

Rule 21.15 — Required · Decidable · Single TU ⚑verify *(AMD1)*

Guideline: The pointer arguments to `memcpy`, `memmove`, `memcmp` must be pointers to compatible types.

Rule 21.16 — Required · Decidable · Single TU ⚑verify *(AMD1)*

Guideline: The pointer arguments to `memcmp` must point to appropriate essential types (not e.g. floating or struct with padding).

Rule 21.17 — Mandatory · Undecidable · Single TU ⚑verify *(AMD1)*

Guideline: Use of `<string.h>` functions must not access beyond the bounds of the referenced objects. **No deviation.**

Rule 21.18 — Mandatory · Undecidable · Single TU ⚑verify *(AMD1)*

Guideline: The `size_t` argument to a `<string.h>` function must have an appropriate value. **No deviation.**

Rule 21.19 — Mandatory · Decidable · Single TU ⚑verify *(AMD1)*

Guideline: Pointers returned by `localeconv`, `getenv`, `setlocale`, `strerror` must be treated as pointer to const.

Rule 21.20 — Mandatory · Decidable · Single TU ⚑verify *(AMD1)*

Guideline: The pointer returned by certain library functions must not be used after a subsequent call to the same function.

Rule 21.21 — Required · Decidable · Single TU ⚑verify *(AMD3)*

Guideline: `system` of `<stdlib.h>` must not be used.

Rule 21.22–21.26 — ⚑verify *(AMD3)*

Guideline: Type-generic math (`<tgmath.h>`) and C11 atomics/memory-ordering/ thread-local constraints.

---

§22 — RESOURCES

Rule 22.1 — Required · Undecidable · System

Guideline: All resources obtained via Standard Library functions must be explicitly released.

```
FILE *f = fopen("x","r"); /* ... */    /* Non-compliant if never fclose(f) */
```

Rule 22.2 — Mandatory · Undecidable · System

Guideline: A block of memory must only be freed if it was allocated by a Standard Library function. **No deviation.**

```
int a[4]; free(a);   /* Non-compliant: a not heap-allocated */
```

Rule 22.3 — Required · Undecidable · System

Guideline: The same file must not be open for read and write on different streams at the same time.

Rule 22.4 — Mandatory · Undecidable · System

Guideline: There must be no attempt to write to a stream opened read-only. **No deviation.**

Rule 22.5 — Mandatory · Undecidable · Single TU

Guideline: A pointer to a `FILE` must not be dereferenced. **No deviation.**

Rule 22.6 — Mandatory · Undecidable · System

Guideline: A `FILE` pointer must not be used after the stream is closed. **No deviation.**

```
fclose(f); fprintf(f,"x");   /* Non-compliant: use after close */
```

Rule 22.7 — Required · Decidable · Single TU ⚑verify *(AMD1)*

Guideline: `EOF` must only be compared with the unmodified return value of a library function capable of returning `EOF`.

Rule 22.8 — Required · Undecidable · System ⚑verify *(AMD1)*

Guideline: `errno` must be set to zero before a call to an errno-setting function.

Rule 22.9 — Required · Undecidable · System ⚑verify *(AMD1)*

Guideline: `errno` must be tested after calling an errno-setting function.

```
errno = 0; v = strtol(s,&e,10); if (errno != 0) { /* handle */ }   /* Compliant */
```

Rule 22.10 — Required · Undecidable · System ⚑verify *(AMD1)*

Guideline: `errno` must only be tested when the last function called was an errno-setting function.

Rule 22.11–22.20 — ⚑verify *(AMD3)*

Guideline: C11 threads/concurrency (`<threads.h>`): mutexes, condition variables, and thread-specific storage — creation, ownership, and release constraints.

MISRA C++

MISRA C++:2023 — Overview

MISRA C++:2023 is a **complete rewrite** of the C++ guidelines, replacing MISRA C++:2008. It targets **ISO C++17** and aligns closely with the AUTOSAR C++14 guidelines (which it effectively absorbs and supersedes).

- **179 guidelines total** = **4 directives** + **175 rules**.
- Each guideline has a **category** and a **decidability**, same scheme as C.
- The **section numbering mirrors the ISO C++ standard's clause structure**, so a

rule ID like `Rule 6.7.2` sits under the C++ clause on the same topic.

CATEGORIES

Same three-tier scheme as MISRA C:

CATEGORY	DEVIATIONS
Mandatory	Never permitted
Required	Permitted only with a formal record
Advisory	Permitted; recording recommended

C++:2023 also classifies each guideline's analysis as **Decidable** or **Undecidable**, and by scope (**Single Translation Unit** or **System**), just like MISRA C.

GUIDELINE ID SCHEME

MISRA C++:2023 rules use a **three-part number** derived from the C++ standard's clause layout: `Rule <clause>.<subclause>.<index>` (e.g. `Rule 8.2.3`). Directives use `Dir <n>.<n>.<n>`. This differs from MISRA C's two-part `Rule S.N` scheme — do not assume a C rule number maps to the same C++ number.

SECTION STRUCTURE (MIRRORS ISO C++17 CLAUSES)

The rules are grouped under sections corresponding to the C++ standard, broadly:

AREA	TYPICAL CLAUSE TOPIC
General / language independent	Character sets, program structure
Lexical conventions	Identifiers, literals, comments
Basic concepts	Declarations, scope, storage duration, ODR
Standard conversions	Implicit conversions, integral/pointer conversions
Expressions	Operators, casts, `sizeof`, order of evaluation
Statements	Selection, iteration, jump, `switch`
Declarations	`const`/`constexpr`, `auto`, type specifiers
Declarators	Pointers, references, function declarators
Classes	Members, access, special member functions
Derived classes	Inheritance, virtual functions
Overloading	Operator/function overloading, conversions
Templates	Template parameters, instantiation, specialization
Exception handling	`throw`, `catch`, `noexcept`, stack unwinding
Preprocessor	Macros, `#include`, conditional compilation
Library introduction / usage	Use of the C++ standard library

> The exact clause names, per-clause rule counts, and every rule ID must be confirmed against the official MISRA C++:2023 document — see the note below.

⚠️ **ENUMERATION STATUS — IMPORTANT**

Unlike the MISRA C bundle (whose rule list is stable and long-established via C:2012), the **verbatim per-rule identifiers of MISRA C++:2023 could not be grounded** in this build: the authoritative reference lists are blocked by the environment's network policy, the standard is copyright-protected, and the reachable community sources use the *old* MISRA C++:2008 hyphenated numbering (`5-0-1`) rather than the 2023 dotted scheme (`8.2.3`), so they are unsafe to copy.

The `cpp/rules/` files therefore provide **comprehensive thematic coverage** — every ISO C++17 clause area, with rationale and compliant/non-compliant examples per guideline — but with each entry marked `⚑verify`. Treat this as a complete **engineering reference**; treat the official standard as the authority for the exact IDs/categories needed in a **formal compliance matrix**.

To pin the exact `Rule x.y.z` identifiers, provide **any one** of:

- the official MISRA C++:2023 PDF (place it in the repo; I'll paraphrase it), or
- a plain-text/CSV list of the rule IDs + titles, or
- an unblocked reference page listing them.

MISRA C++

MISRA C++:2023 — Directives

MISRA C++:2023 defines **4 directives** — guidance requiring information beyond the source code (process, documentation, design). The exact IDs and wording below should be confirmed against the official standard (`⚑verify`).

- **Dir (implementation behaviour)** — *(Required)* ⚑verify — Any use of

implementation-defined behaviour on which the program's output depends must be documented and understood. *(analogue of MISRA C Dir 1.1)*

- **Dir (traceability)** — *(Required)* ⚑verify — Code should be traceable to

documented requirements. *(analogue of MISRA C Dir 3.1)*

- **Dir (run-time failure)** — *(Required)* ⚑verify — Run-time failures should be

minimized / handled. *(analogue of MISRA C Dir 4.1)*

- **Dir (documentation of design intent)** — *(Advisory)* ⚑verify — Design intent

that cannot be expressed in code (e.g. rationale for permitted deviations, encapsulation choices) should be documented.

> These four map conceptually to the MISRA C directive families but the official C++:2023 numbering and precise text differ. Verify before formal use.

MISRA C++

MISRA C++:2023 — Rules (comprehensive, in depth)

> **Coverage & accuracy.** This file gives **comprehensive thematic coverage** of MISRA C++:2023, organized under the **ISO C++17 clause areas** the standard's section numbering mirrors, with rationale and compliant/non-compliant C++ for each guideline. Every entry is marked `⚑verify`: the *guidance* reflects long-established MISRA C++ / AUTOSAR C++14 practice, but the **exact rule IDs, categories, and official wording of MISRA C++:2023 must be confirmed against the purchased standard** — its per-rule identifiers (dotted, e.g. `Rule 8.2.3`) could not be grounded in this build environment (authoritative lists are copyright- protected / network-blocked). Use this as an engineering reference; use the official document for a formal compliance matrix. See `../overview.md`.

Legend: category shown as *(Mandatory / Required / Advisory)*.

---

1. GENERAL & LEXICAL CONVENTIONS

- **Trigraphs and digraphs must not be used.** *(Required)* ⚑verify
- **The `/*` and `//` sequences must not appear within a comment.** *(Required)* ⚑verify

```
class="c">/* a /* b */ c;   class="c">// Non-compliant: 'c;' is live code
```

- **Each expression statement / declaration should be on its own line.** *(Advisory)* ⚑verify
- **Literal suffixes must be uppercase (`L`, `U`, `LL`).** *(Required)* ⚑verify

```
auto n = 100l;   class="c">// Non-compliant     auto n = 100L;  // Compliant
```

- **Octal constants (other than `0`) and octal escape sequences must not be used.** *(Required)* ⚑verify
- **User-defined literals should be used only where they improve clarity.** *(Advisory)* ⚑verify
- **A character/string literal must not be modified.** *(Required)* ⚑verify

2. IDENTIFIERS & NAMING

- **Identifiers must not be reused across scopes so as to shadow/hide.** *(Required)* ⚑verify

```
int count;
void f(){ int count = 0; }   class="c">// Non-compliant: hides outer 'count'
```

- **An identifier must not be reused for different entities in different name spaces confusingly.** *(Required)* ⚑verify
- **Identifiers should be distinct in a typographically unambiguous way** (avoid `l`/`1`, `O`/`0`). *(Advisory)* ⚑verify
- **Macro names must be reserved-safe and distinct from other identifiers.** *(Required)* ⚑verify

3. BASIC CONCEPTS (DECLARATIONS, SCOPE, ODR, STORAGE DURATION, LIFETIME)

- **Every entity must have exactly one definition (ODR).** *(Required)* ⚑verify
- **An object must be initialized before its value is read.** *(Mandatory)* ⚑verify

```
int x; use(x);        class="c">// Non-compliant
int x{0}; use(x);     class="c">// Compliant (brace-init also prevents narrowing)
```

- **Declare each variable in the narrowest practical scope, initialized at declaration.* (Advisory)* ⚑verify
- **The static initialization-order fiasco must be avoided** (no cross-TU dependency between dynamically-initialized namespace-scope objects). *(Required)* ⚑verify

```
class="c">// Compliant: function-local static, initialized on first use
Config& config(){ static Config c; return c; }
```

- **An object's address must not be used outside its lifetime** (no dangling). *(Required)* ⚑verify
- **`thread_local` storage must be used deliberately and documented.** *(Advisory)* ⚑verify

4. STANDARD CONVERSIONS

- **Implicit conversions that lose value/precision (narrowing) must not occur.** *(Required)* ⚑verify

```
int i = 300; char c = i;             class="c">// Non-compliant: narrowing
char c{ static_cast<char>(i) };      class="c">// Compliant: explicit
```

- **`const`/`volatile` qualification must not be cast away.** *(Required)* ⚑verify
- **Pointer↔integer conversions should be avoided.** *(Advisory)* ⚑verify
- **A null pointer must be written `nullptr`, never `0` or `NULL`.** *(Required)* ⚑verify

```
int* p = NULL;      class="c">// Non-compliant
int* p = nullptr;   class="c">// Compliant
```

- **An array must not decay to a pointer where the size is needed** (prefer `std::array` / `std::span`). *(Advisory)* ⚑verify

5. EXPRESSIONS

- **Operator precedence should be made explicit with parentheses.** *(Advisory)* ⚑verify
- **The comma operator must not be used.** *(Required)* ⚑verify
- **Evaluation must not depend on unspecified order of evaluation.** *(Required)* ⚑verify

```
y = f() + g();   class="c">// Non-compliant if f/g share mutable state
```

- **C-style and functional-notation casts must not be used; use named casts.** *(Required)* ⚑verify

```
double d = (double)i;                class="c">// Non-compliant
double d = static_cast<double>(i);   class="c">// Compliant
```

- **`reinterpret_cast` must not be used** (save rare, documented low-level cases). *(Required)* ⚑verify
- **`const_cast` must not be used to modify a const object.** *(Required)* ⚑verify
- **`dynamic_cast` result must be checked before use.** *(Required)* ⚑verify
- **The operand of `sizeof` must not have side effects.** *(Mandatory)* ⚑verify
- **Bitwise operators must not be applied to signed/Boolean operands.** *(Required)* ⚑verify
- **Increment/decrement must not be mixed with other side effects in one expression.* (Advisory)* ⚑verify
- **The result of an assignment must not be used as a sub-expression.** *(Advisory)* ⚑verify

```
if ((x = get()) != 0) {}   class="c">// Non-compliant
```

6. STATEMENTS (CONTROL FLOW)

- **The body of an `if`/`else`/loop must be a compound statement `{ }`.** *(Required)* ⚑verify
- **An `if...else if` chain must end with an `else`.** *(Required)* ⚑verify
- **Every `switch` must have a `default`; each clause ends with `break`** (no unintended fall-through). *(Required)* ⚑verify
- **A `switch` must have at least two case-clauses.** *(Required)* ⚑verify
- **`goto` must not be used** (or only jump forward within the same function). *(Advisory/Required)* ⚑verify
- **A loop counter must not be of floating type.** *(Required)* ⚑verify
- **A `for` loop must be well-formed; range-based `for` is preferred.** *(Advisory)* ⚑verify

```
for (const auto& item : items) { use(item); }   class="c">// Compliant
```

- **A function should have a single exit at the end (or use RAII to make multiple exits safe).* (Advisory)* ⚑verify

7. DECLARATIONS (SPECIFIERS)

- **Types must be explicitly specified where `auto` would harm clarity/safety.** *(Advisory)* ⚑verify
- **Use `const`/`constexpr` for values that do not change.** *(Advisory)* ⚑verify

```
constexpr int kMax = 100;   class="c">// Compliant
```

- **`volatile` must be used only for genuinely volatile access.** *(Required)* ⚑verify
- **A declaration must declare no more than one variable.** *(Advisory)* ⚑verify
- **`using` (type alias) is preferred over `typedef`.** *(Advisory)* ⚑verify
- **An enumeration should be scoped (`enum class`).** *(Required)* ⚑verify

```
enum Color { Red };          class="c">// Non-compliant: unscoped
enum class Color { Red };    class="c">// Compliant
```

- **`[[nodiscard]]` should mark functions whose result must be used.** *(Advisory)* ⚑verify

8. DECLARATORS (POINTERS, REFERENCES, FUNCTIONS, ARRAYS)

- **A non-`void` function must return a value on every path.** *(Mandatory)* ⚑verify
- **Variable-length arrays must not be used.** *(Required)* ⚑verify
- **Pointer arithmetic must remain within one array; prefer `std::span`/iterators.** *(Required)* ⚑verify
- **The address of an automatic object must not outlive it.** *(Required)* ⚑verify
- **Functions must not be recursive (bounded stack).** *(Required)* ⚑verify
- **A function must not have an unbounded variadic (`...`) parameter list;** use variadic templates instead. *(Required)* ⚑verify
- **Function parameters should be `const` references for non-trivial types.** *(Advisory)* ⚑verify
- **Default arguments must not be repeated/contradicted across declarations.** *(Required)* ⚑verify
- **A parameter should not be modified within the function.** *(Advisory)* ⚑verify

9. CLASSES & SPECIAL MEMBER FUNCTIONS

- **A class managing a resource must obey the Rule of 0/3/5.** *(Required)* ⚑verify

```
struct Bad { int* p; ~Bad(){ delete p; } };   class="c">// Non-compliant: copy => dou
```

- **A polymorphic base class must have a `virtual` (or protected non-virtual)

destructor.* (Required)* ⚑verify

- **Data members should be `private`.** *(Advisory)* ⚑verify
- **Single-argument constructors should be `explicit`.** *(Required)* ⚑verify
- **Members must be initialized in declaration order (member-init list).** *(Required)* ⚑verify
- **A constructor must not call a virtual function that expects a fully

constructed object.* (Required)* ⚑verify

- **`= default` / `= delete` should express intent for special members.** *(Advisory)* ⚑verify
- **A class with virtual functions should mark them `override` / `final` in

derived classes.* (Required)* ⚑verify

10. DERIVED CLASSES & INHERITANCE

- **Every override must be marked `override`.** *(Required)* ⚑verify

```
struct D : Base { void f() override; };   class="c">// Compliant
```

- **A virtual function's default arguments must not be changed in an override.** *(Required)* ⚑verify
- **Multiple inheritance from non-interface (stateful) bases should be avoided.** *(Advisory)* ⚑verify
- **Virtual inheritance should be used only where a diamond truly requires it.** *(Advisory)* ⚑verify
- **A base class subobject must not be sliced by value copy.** *(Required)* ⚑verify

11. OVERLOADING

- **Overloaded operators must preserve conventional semantics** (`==` symmetric,

side-effect-free, etc.). *(Required)* ⚑verify

- **Overload sets must not be ambiguous or rely on surprising conversions.** *(Required)* ⚑verify
- **`operator new`/`operator delete` must be provided and used in matching pairs.** *(Required)* ⚑verify
- **User-defined conversion operators should be `explicit` unless implicit

conversion is intended.* (Advisory)* ⚑verify

12. TEMPLATES & GENERIC PROGRAMMING

- **A template must be well-formed for every permitted instantiation.** *(Required)* ⚑verify
- **Explicit specializations must be declared before first use.** *(Required)* ⚑verify
- **Template parameters should be constrained (concepts / `static_assert`).** *(Advisory)* ⚑verify

```
static_assert(std::is_integral_v<T>, "T must be integral");   class="c">// Compliant
```

- **SFINAE / overload resolution must not silently select an unintended

specialization.* (Required)* ⚑verify

- **Non-type template parameters must have appropriate types.** *(Required)* ⚑verify

13. EXCEPTION HANDLING

- **Exceptions must not escape a `noexcept` function or a destructor.** *(Required)* ⚑verify

```
~Widget() noexcept { class="c">/* must not throw */ }
```

- **Exceptions must be caught by (const) reference, not by value.** *(Required)* ⚑verify

```
catch (const std::exception& e) { }   class="c">// Compliant
catch (std::exception e) { }          class="c">// Non-compliant: slicing
```

- **Only objects derived from a common exception base should be thrown.** *(Required)* ⚑verify

- **An empty `throw;` (rethrow) must appear only inside a `catch`.** *(Required)* ⚑verify
- **Every exception thrown must have a matching handler on all paths.** *(Required)* ⚑verify
- **`catch(...)` should be used only as a last-resort/boundary handler.** *(Advisory)* ⚑verify
- **Resources must be released on the exceptional path (RAII, not manual cleanup).** *(Required)* ⚑verify

14. DYNAMIC MEMORY & RESOURCE MANAGEMENT

- **Dynamic allocation must be constrained/avoided; when used, ownership must be explicit via RAII (smart pointers).* (Required)* ⚑verify

```
auto p = std::make_unique<Widget>();   class="c">// Compliant
Widget* p = new Widget();              class="c">// Non-compliant: manual delete
```

- **Raw owning pointers must not be used.** *(Required)* ⚑verify
- **`new`/`delete` (and `new[]`/`delete[]`) must be correctly paired.** *(Required)* ⚑verify
- **A pointer must not be used after the object is freed / moved-from.** *(Mandatory)* ⚑verify
- **`std::move` must not be used on a `const` object or a value used afterwards.** *(Required)* ⚑verify

15. CONCURRENCY

- **Shared data accessed by multiple threads must be protected (mutex/atomic).** *(Required)* ⚑verify
- **A mutex must be released on all paths (`std::lock_guard`/`scoped_lock`).** *(Required)* ⚑verify

```
std::lock_guard<std::mutex> lk(m);   class="c">// Compliant: RAII unlock
```

- **Deadlock must be avoided (consistent lock ordering / `std::scoped_lock`).** *(Required)* ⚑verify
- **A thread must be joined or detached before its handle is destroyed.** *(Required)* ⚑verify

- **Data races (unsynchronized concurrent access) are undefined and must not occur.* (Mandatory)* ⚑verify

16. PREPROCESSOR

- **Function-like macros should be replaced by inline / `constexpr` functions.** *(Advisory)* ⚑verify

```
#define SQ(x) ((x)*(x))                    class="c">// Non-compliant (advisory)
constexpr int sq(int x){ return x * x; }   class="c">// Compliant
```

- **Object-like macros for constants should be `constexpr` / `enum` instead.** *(Advisory)* ⚑verify
- **Macro parameters in the replacement list must be parenthesised.** *(Required)* ⚑verify
- **`#include` operands must be well-formed** (no `'`, `"`, `\`, `/*`, `//`). *(Required)* ⚑verify
- **Headers must have include guards or `#pragma once`.** *(Required)* ⚑verify
- **`#undef` should not be used.** *(Advisory)* ⚑verify
- **`#` / `##` operators should not be used.** *(Advisory)* ⚑verify

17. STANDARD LIBRARY USAGE

- **Deprecated or removed library features must not be used.** *(Required)* ⚑verify
- **C standard-library facilities superseded by C++ equivalents should not be used* *(`<cstring>` raw ops → `std::string`/`std::array`/algorithms).* (Advisory)* ⚑verify

```
std::array<int,4> a{};   class="c">// Compliant, over C array + memset
```

- **Library functions must be called with valid arguments (ranges, bounds).** *(Required)* ⚑verify
- **Iterators must not be used after invalidation.** *(Mandatory)* ⚑verify
- **`std::vector<bool>` and other surprising specializations should be avoided.** *(Advisory)* ⚑verify
- **C++ I/O / locale features with heavy runtime cost should be used deliberately.* (Advisory)* ⚑verify

- **`<csetjmp>`, `<csignal>`, `<cstdarg>` facilities must not be used.**
  (Required) ⚑verify

Directives

See `directives.md` for the four MISRA C++:2023 directives.

Generated from the `misra-guidelines` skill · paraphrased reference, not the official MISRA standard.